Case Study: Traditional Keyword Search vs AI-Powered Semantic Search Using ChromaDB

A PROJECT REPORT



Submitted by

ISHAAN SAHU RA2311003010352
PRATEEK SINHA RA2311003010318
ARNAV AMRIT RA2311003010299
APEKSHA SACHAN RA2311003010306
SAVIO BINO RA2311003010291

Under the guidance of

[Project Guide Name]

[Designation and Department]

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTING TECHNOLOGIES
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203

BONAFIDE CERTIFICATE

Certified that the project report entitled “Case Study: Traditional Keyword Search vs AI-Powered Semantic Search Using ChromaDB” is the bona fide work of the following students, who carried out the project work under the guidance of [Project Guide Name and Designation]:

- ISHAAN SAHU [RA2311003010352]
- PRATEEK SINHA [RA2311003010318]
- ARNAV AMRIT [RA2311003010299]
- APEKSHA SACHAN [RA2311003010306]
- SAVIO BINO [RA2311003010291]

This report is submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering. The declaration and signatures below are to be completed by the project guide and the department before formal submission.

_____	_____
PROJECT GUIDE Name and designation	HEAD OF THE DEPARTMENT Department of Computing Technologies

ACKNOWLEDGEMENTS

The project team acknowledges the Department of Computing Technologies, SRM Institute of Science and Technology, for the academic setting in which this work was undertaken. The team also thanks its project guide and faculty reviewers for their feedback during project development. The guide's name and any additional acknowledgements should be completed by the authors before formal submission.

ABSTRACT

This project develops Semantic Search Studio, a local document-search application and a case study comparing BM25 keyword retrieval with embedding-based semantic retrieval over the same text passages. The application accepts PDF, DOCX, and TXT files, extracts text, creates sentence-aware word-count chunks, stores metadata and judgments in SQLite, and stores normalized sentence embeddings in persistent ChromaDB. Users can search by exact terms or meaning, run paired comparisons, record graded relevance judgments, inspect Precision@K, Recall@K, mean reciprocal rank (MRR), normalized discounted cumulative gain (NDCG@K), and review retrieval-only and end-to-end latency.

The evaluation workflow is designed to hold the corpus, chunk boundaries, query, and top-K cutoff constant across retrieval methods. It records the model and experiment configuration, repeated timing variation, and relevance-judgment coverage. The project also documents limitations that affect interpretation, including subjective labels, incomplete judgments, a small or unrepresentative corpus, absent OCR for scanned PDFs, and latency scopes that exclude model loading and shared SQLite chunk loading.

At the time of this report, the local project database contains no indexed documents, search queries, relevance judgments, or completed experiments. The software and evaluation workflow are present in the repository, but no comparative quality or performance result can yet be reported. The next study phase is to select a representative corpus, define and categorize queries, judge all corpus chunks, run paired experiments, and update this report with the exported measurements.

Keywords: Information retrieval, BM25, semantic search, sentence embeddings, ChromaDB, relevance evaluation

TABLE OF CONTENTS

ACKNOWLEDGEMENTS iii
ABSTRACT iv
TABLE OF CONTENTS v
LIST OF FIGURES vi
LIST OF TABLES vii
ABBREVIATIONS viii
CHAPTER 1 INTRODUCTION 1
CHAPTER 2 BACKGROUND AND LITERATURE SURVEY 3
CHAPTER 3 SYSTEM DESIGN AND EXECUTION METHODOLOGY 5
CHAPTER 4 IMPLEMENTATION AND DATA DESIGN 8
CHAPTER 5 QUERIES AND EXPERIMENTAL METHODOLOGY 10
CHAPTER 6 RESULTS AND DISCUSSION 12
CHAPTER 7 CONCLUSION AND FUTURE ENHANCEMENT 14
REFERENCES 15
APPENDIX A PROJECT SETUP AND CONFIGURATION 16
APPENDIX B STARTER QUERY EXAMPLES 17
APPENDIX C SOURCE AND TEST INVENTORY 18

LIST OF FIGURES

Figure	Title	Page
Figure 3.1	System architecture and data flow	5
Figure 3.2	Guided workflow from document upload to evaluation	7

LIST OF TABLES

Table	Title	Page
Table 3.1	Functional requirements and implementation status	6
Table 3.2	Technology stack and role	6
Table 3.3	Persistent data entities	7
Table 4.1	Default project configuration	9
Table 5.1	Evaluation metrics	11
Table 5.2	Latency scopes	11
Table 6.1	Current implementation and evaluation status	12
Table 6.2	Functional verification summary	12
Table B.1	Starter query examples supplied with the project	18
Table C.1	Automated test modules in the repository	19

ABBREVIATIONS

Term	Meaning
BM25	Okapi probabilistic term weighting and ranking function
CPU	Central Processing Unit
CSV	Comma-Separated Values
IDF	Inverse Document Frequency
IR	Information Retrieval
MRR	Mean Reciprocal Rank
NDCG	Normalized Discounted Cumulative Gain
PDF	Portable Document Format
SQLite	Embedded relational database engine
TF	Term Frequency
UI	User Interface

CHAPTER 1
INTRODUCTION

Case-study context and research objectives

1.1 Introduction to the project

Information retrieval systems rank documents or passages in response to a query. Traditional lexical ranking methods such as BM25 use the occurrence and distribution of query terms, with document-length normalization. They provide a strong baseline when a query contains the exact names, identifiers, and technical terms present in a passage [1]. Dense sentence embeddings offer a complementary approach: a model maps text to vectors so that passages with related wording can be retrieved by vector distance [2].

This project packages that comparison as Semantic Search Studio, a local document-search workspace. A user uploads a small corpus, searches it with BM25 or with a Sentence Transformers model, and can compare both ranked lists. A separate evaluation workflow lets the user label passages and compute ranking metrics. The academic case study asks where the two methods perform well and where they fail; it does not assume that semantic retrieval is always better.

1.2 Problem statement

The project addresses the difficulty of making a fair, repeatable comparison between exact-term retrieval and meaning-based retrieval over a user-provided document collection. A valid comparison needs the same extracted corpus, passage boundaries, queries, and top-K cutoff for both methods, as well as explicit relevance judgments and timing definitions. Without those controls, differences in results can reflect different input data or measurement scopes rather than the retrieval method.

1.3 Motivation

BM25 and vector retrieval respond to different signals. BM25 can favor a passage containing a rare identifier or exact phrase, while an embedding model can retrieve a passage that expresses an idea with different words. Showing both methods side by side makes these trade-offs visible. Keeping the application local also lets users work with private documents without sending corpus text to a hosted search service.

1.4 Objectives

- Build a local interface for uploading and managing PDF, DOCX, and TXT files.
- Extract and chunk text while retaining source names and available PDF page numbers.
- Index the same passage set for BM25 and normalized sentence-embedding retrieval.
- Compare method-specific rankings, scores, and clearly named timing scopes.
- Support categorized queries, explicit 0-3 relevance grades, and paired evaluation using standard ranking metrics.
- Persist experiment configuration and raw per-query outputs so findings can be reviewed and exported.

1.5 Research questions

1. How do BM25 and embedding-based retrieval differ in ranking quality on the same local document passages and query set?
2. How do retrieval-only and end-to-end query timings differ when each scope is measured consistently?
3. Which query categories, such as paraphrases, exact phrases, identifiers, or domain terms, show different retrieval behavior?

1.6 Scope and boundaries

The implemented scope includes local file ingestion, text extraction, word-count chunking, BM25 ranking, Sentence Transformers embeddings, persistent ChromaDB storage, paired search, manual relevance labels, metric calculation, experiment history, and CSV export. OCR, a hosted language model, generated answers, hybrid ranking, and a production multi-user deployment are outside the current implementation. The separate RDFLib triple demonstration is educational and does not participate in the retrieval pipeline.

1.7 Organization of the report

Chapter 2 reviews the retrieval concepts and evaluation practices used to frame the comparison. Chapter 3 describes requirements, architecture, persistence, and design controls. Chapter 4 documents the implementation. Chapter 5 defines the experiment protocol. Chapter 6 records the current implementation and evidence status without inventing results. Chapter 7 summarizes the work and lists the next study steps. The references and appendices follow the chapters.

CHAPTER 2
BACKGROUND AND LITERATURE SURVEY

Retrieval models, embeddings, vector search, and evaluation

2.1 Lexical retrieval and BM25

BM25 is a probabilistic term-weighting model that scores documents according to query-term matches, term frequency, inverse document frequency, and document length. Its saturation and length-normalization parameters help prevent repeated terms or long passages from dominating without limit [1]. In this implementation, the defaults are k1 = 1.5 and b = 0.75. The score is a ranking value: it is not a probability, percentage, or confidence estimate.

The project tokenizer lowercases Unicode text and extracts word-like tokens. For each query term found in a chunk, the engine computes a positive inverse-document-frequency contribution and combines it with normalized term frequency. Chunks with no matching query term receive no BM25 result. This design makes the baseline interpretable and useful for exact names and vocabulary, while leaving language-specific stemming and synonym expansion out of scope.

2.2 Dense sentence embeddings

Sentence-BERT uses a siamese or triplet network structure to produce sentence embeddings that can be compared with cosine similarity, making it practical to encode query and passage separately [2]. The application uses the all-MiniLM-L6-v2 model through Sentence Transformers. Its model card describes 384-dimensional sentence embeddings and semantic-search use cases [3]. The wrapper normalizes vectors before storing or querying them.

A vector distance is a geometric measure of closeness in the model space. The application asks ChromaDB for cosine distance; lower distance ranks closer vectors first. That number is not a calibrated relevance probability. The model can also miss exact rare terms, identifiers, negation, or domain distinctions, so results need comparison and human judgments.

2.3 Vector databases and ChromaDB

A vector database associates each record with an identifier, text, metadata, and embedding. Chroma collections provide this organization and support nearest-neighbor queries. The current application creates a persistent collection with cosine space, stores passage text and source metadata, and returns distances for ranking [4]. Persistence allows the local collection to survive app restarts.

The implementation configures cosine space through Chroma's legacy `hnsw:space` collection metadata key. Current Chroma documentation describes a newer collection configuration interface and identifies the metadata form as a legacy path [5]. This compatibility detail should be revisited when the supported Chroma version is pinned or upgraded.

2.4 Relevance judgments and ranking metrics

Retrieval quality depends on relevance labels. The Evaluation page lets the user assign each chunk a grade from 0 (not relevant) through 3 (highly relevant), or leave it explicitly unjudged. The project

records the number of judged chunks and coverage. The TREC evaluation ecosystem includes precision, recall, reciprocal-rank, and discounted-gain measures for comparing ranked outputs [6].

Graded measures such as NDCG preserve differences among marginally and highly relevant passages, while MRR focuses on the position of the first relevant result. These measures answer different questions and should be presented together with the chosen cutoff, query set, judgment protocol, and coverage. In this implementation, unjudged chunks are treated as grade 0 during metric computation. Consequently, incomplete judgments can bias results and must be disclosed.

2.5 Embedding similarity and symbolic semantics

The term semantic is used in two different technical senses in the repository. Dense embeddings model distributional relationships in language and support vector similarity. RDF and OWL instead represent explicit symbols, relations, classes, and constraints; an ontology reasoner can infer consequences from those declarations. The RDFLib sample demonstrates a triple separately. Neither ChromaDB nor the embedding index is an RDF store or an OWL reasoner.

2.6 Position of this case study

The project is an applied, controlled comparison tool rather than a claim that one retrieval family dominates the other. Its contribution is a transparent workflow that keeps the corpus and query conditions aligned, records graded judgments and configuration, and exposes both ranking and latency evidence for inspection. The quality of its eventual conclusions will depend on the chosen corpus, query design, label coverage, and the software and hardware environment recorded during the experiment.

CHAPTER 3
SYSTEM DESIGN AND EXECUTION METHODOLOGY

Requirements, architecture, data flow, and comparison controls

3.1 System overview

Semantic Search Studio is a single-machine Streamlit application. The interface accepts user documents and queries, the service layer coordinates retrieval and timing, the core modules implement extraction and ranking, SQLite stores relational records, and ChromaDB stores vectors and passage metadata. The default embedding model loads lazily on the first semantic request. The overall data flow is shown in Figure 3.1.

Figure 3.1: System architecture and data flow

PDF, DOCX, and TXT uploads
Text extraction and source metadata
Whitespace normalization and sentence-aware chunking
SQLite: document records, passage text, queries, judgments, history
BM25 over SQLite passages **Local embeddings in ChromaDB**
Paired ranked results with source, score, and timing
Human relevance grades and paired evaluation
Precision@K, Recall@K, MRR, NDCG@K, latency, and CSV export

3.2 Functional requirements

Table 3.1: Functional requirements and implementation status

Requirement	Implementation evidence	Status
Upload local documents	Streamlit uploader accepts PDF, DOCX, and TXT; size cap is configurable.	Implemented
Extract searchable text	PyMuPDF, python-docx, and UTF-8 text paths; scanned PDFs report that OCR is unavailable.	Implemented with OCR limit
Create repeatable passages	Sentence-aware chunker uses configurable word-count size and overlap.	Implemented
Run two retrieval methods	BM25 and Chroma cosine retrieval use the same stored chunk set.	Implemented
Compare results	Paired comparison runs both methods for one query and top-K value.	Implemented
Judge and evaluate	Graded labels, query categories, metrics, latency repeats, and coverage are persisted.	Implemented; study data pending
Export findings	Selected experiment can be exported as CSV.	Implemented; no experiment yet

3.3 Non-functional requirements

- Local operation: uploaded files, metadata, and vector data are stored on the configured local data path.
- Reproducibility: model, chunk settings, top K, latency repeats, distance metric, and corpus size are saved with each experiment.
- Interpretability: method labels and score directions are shown in the interface; score values are not presented as probabilities.
- Portability: the setup targets Windows and Python 3.10 or 3.11, with CPU inference and optional CUDA.
- Data integrity: ordinary ingestion failures trigger cleanup across stores, but SQLite and ChromaDB cannot share one atomic transaction.

3.4 Technology stack

Table 3.2: Technology stack and role

Technology	Role in the application
Python	Application logic, text processing, ranking, and experiment orchestration.
Streamlit	Local browser interface, guided navigation, forms, tables, and charts.
Sentence Transformers	Loads and runs the local all-MiniLM-L6-v2 embedding model.
ChromaDB	Persists normalized passage vectors and returns cosine distances.
SQLite	Stores document records, chunks, history, queries, labels, experiments, and scores.
PyMuPDF and python-docx	Extract PDF pages and DOCX paragraphs and tables.
Plotly and pandas	Display evaluation charts and tabular records.
RDFLib	Runs a separate demonstration of a subject-predicate-object triple.

3.5 Data persistence

The configured data directory contains originals under uploads, a persistent Chroma directory, and an SQLite database. SQLite stores document metadata and passage text, while Chroma stores embeddings, text, and metadata. The stores are intentionally separate. Deleting a document removes Chroma records and cascades the relational passage and judgment records. Since there is no distributed transaction between the two stores, a process interruption during indexing can require deletion and re-indexing.

Table 3.3: Persistent data entities

Entity	Purpose
documents	File identity, name, type, timestamp, status, page/chunk totals, hash, and error.
chunks	Passage identifiers, source document, order, optional page number, and extracted text.
search_history	Query, method, time, result count, saved results, and timing values.
test_queries and relevance_judgments	Query category and expected information, plus explicit per-chunk grades.

Entity	Purpose
experiments and evaluation_results	Experiment configuration and per-query, per-method metrics, timing variation, and judgment coverage.

3.6 Guided user workflow

The interface begins with an overview and a left navigation drawer. The product workflow progresses through document ingestion, a first search, a paired method comparison, and the Evaluation workbench. Later workflow pages become available after the preceding action has been recorded. Search History and Settings remain supporting workspace pages. Figure 3.2 summarizes the intended sequence.

Figure 3.2: Guided workflow from document upload to evaluation

01 Add documents and build the local library
02 Run a keyword or semantic query
03 Compare both ranking methods on the same query
04 Create test queries, grade passages, and run a paired experiment
Review history, configuration, metric charts, and CSV output

CHAPTER 4
IMPLEMENTATION AND DATA DESIGN

The current software components and their behavior

4.1 Document ingestion

The upload interface accepts PDF, DOCX, and TXT files and enforces a configurable per-file size limit. SHA-256 identifies identical file contents and prevents accidental duplicate indexing. PDF extraction reads page text with PyMuPDF and retains one-based page numbers. DOCX extraction joins non-empty paragraphs and table cells. TXT files are read as UTF-8 with a byte-order mark removed and malformed bytes replaced. A scanned PDF with no extractable text produces an explicit OCR limitation message.

4.2 Normalization and chunking

The chunker replaces null characters, collapses whitespace, and splits text into sentence-like units. It builds chunks using a default window of 500 whitespace-delimited words with 75 words of overlap. Oversized individual sentences are windowed. These are word counts rather than a tokenizer-specific token limit. Each chunk receives a stable document-scoped identifier, chunk index, source filename, timestamp, and available page number.

4.3 Semantic indexing

The default model is `all-MiniLM-L6-v2`, which produces 384-dimensional embeddings according to its model card [3]. The embedding engine is lazy: it loads the model on first use, encodes passages in batches, and requests normalized vectors. ChromaDB stores the vectors and source metadata in a persistent collection configured for cosine distance. A query is encoded with the same model before nearest-neighbor search. The interface reports raw cosine distance and explains that lower values rank closer vectors first.

4.4 Keyword retrieval

The BM25 engine reads the indexed SQLite chunks, tokenizes Unicode word sequences in lowercase, and calculates document frequency and average passage length for the collection. The current defaults are k1 = 1.5 and b = 0.75. It scores matching query terms and returns up to the requested K passages in descending score order. The index is constructed in memory for a keyword search, and the end-to-end timing includes this construction plus scoring.

4.5 Search orchestration and comparison

The search service routes a request to BM25 or semantic retrieval and packs the returned passages for history storage. The comparison page runs both methods with the same query and K, then places the ranked outputs side by side. Each result includes the source, passage text, and method-specific score. The comparison records distinct history labels for both arms so that the workflow can recognize a completed paired search.

4.6 Relevance judgments and metrics

Users create categorized queries and assign each indexed chunk a grade from 0 to 3. The data editor also supports an Unjudged state. Saving judgments records only explicit grades. A paired experiment runs each saved query against both methods, computes metrics over the top K, and stores the configuration, timing means and standard deviations, judgment coverage, and per-method scores. Recall is undefined when no relevant items have been labeled. The current metric implementation treats an unjudged chunk as grade 0; coverage therefore matters when interpreting results.

4.7 User interface and exports

The Streamlit interface contains Overview, Document Library, Upload Documents, Semantic Search, Keyword Search, Compare Search, Evaluation, Search History, and Settings. The current redesign uses a warm neutral theme, a collapsible left drawer, glass-like surfaces, and a staged workflow. Streamlit reruns the application as users interact with widgets; the project uses session state and callbacks to preserve the current workspace page and move users through the workflow [7]. Evaluation results can be filtered by query category, plotted with Plotly, and exported as CSV for independent analysis.

4.8 Configuration

Table 4.1: Default project configuration

Setting	Default	Effect
Embedding model	all-MiniLM-L6-v2	Local sentence vectors; 384 dimensions.
Chunk size	500 words	Maximum default word window per passage.
Chunk overlap	75 words	Shared tail words between adjacent chunks.
Default top K	5	Number of results requested per method.
Latency repeats	3	Number of timing runs used in paired evaluation.
Maximum upload	50 MB	Per-file upload limit.
Vector distance	Cosine	Lower raw distance ranks closer vectors.

Settings can be overridden with environment variables documented in `config.py` and `.env.example`. Changing the embedding model or chunk settings requires re-indexing the collection. The experiment configuration records the model and chunk parameters used, along with K and the timing repeat count.

CHAPTER 5
QUERIES AND EXPERIMENTAL METHODOLOGY

A reproducible protocol for the future paired evaluation

5.1 Corpus selection

The study should use a representative, legally usable collection of source documents whose content matches the intended search domain. The same files must be indexed once and used by both retrieval methods. The report should record the corpus source, file types, number of documents, total indexed chunks, language, date of preparation, and any exclusions. The repository includes a short network-security sample for a basic demonstration; it is not a large or representative evaluation corpus.

At report preparation, the local database has zero indexed documents and zero chunks, and the uploads directory is empty. The sample file in `samples/` has not been counted as an indexed evaluation corpus. Corpus size and composition remain to be established by the study team.

5.2 Query design

Queries should cover different retrieval conditions, including paraphrases, exact wording, identifiers, and domain terminology. Each query should have a category and a short description of the information that would satisfy it. The starter CSV supplies three example prompts, but they are instructional examples rather than completed study queries. Queries should be reviewed for clarity and should not be written only to favor one method.

5.3 Relevance protocol

1. Freeze the document corpus and chunk configuration before judging. If either changes, recheck the affected judgments.
2. For each query, inspect every corpus chunk and assign a grade: 0 not relevant, 1 marginally relevant, 2 relevant, or 3 highly relevant.
3. Leave a chunk Unjudged only when the assessor has not evaluated it; do not treat an incomplete judgment set as full corpus truth.
4. Record who judged the passages and agree on examples of each grade before the formal study. If multiple assessors participate, document how disagreements are handled.
5. Save the judgments before running the paired experiment and report the coverage for each query.

5.4 Retrieval controls

For each experiment, both methods must receive the same query string, corpus, chunk boundaries, and top-K value. BM25 uses the configured tokenizer and parameters. Semantic retrieval uses the same configured sentence model for all indexed passages and queries, and the collection uses cosine distance. The model name, collection size, chunk size, overlap, K, and software configuration should be retained with the exported report.

5.5 Metrics

Table 5.1: Evaluation metrics

Metric	Interpretation	Implementation note
Precision@K	Share of the top K retrieved passages that have a positive relevance grade.	Denominator is configured K, including when fewer than K results are returned.
Recall@K	Share of known relevant passages retrieved in the top K.	Undefined when no relevant passage has been labeled.
MRR	Reciprocal rank of the first relevant retrieved passage.	Zero when no relevant passage appears in the top K.
NDCG@K	Ranking quality that rewards highly graded passages near the top.	Uses gain 2^grade - 1 with logarithmic rank discount.
Latency	Observed time for the stated retrieval or end-to-end scope.	Repeated runs are summarized by mean and population standard deviation.

5.6 Timing protocol

The application reports two timing scopes. Retrieval-only timing covers BM25 scoring or the Chroma query. End-to-end query timing covers BM25's per-query in-memory index build plus scoring, or query embedding plus Chroma retrieval. Model loading and shared SQLite chunk loading are excluded from both. Document extraction and index construction are not online query timings. Reports should name both scopes and the hardware, operating system, Python and library versions, and any CPU or CUDA setting.

Table 5.2: Latency scopes

Scope	Included operations	Excluded operations
Retrieval-only	BM25 score calculation or Chroma query execution.	Model loading, query embedding, shared SQLite chunk loading, document indexing.
End-to-end query	BM25 in-memory index build plus scoring; or query embedding plus Chroma retrieval.	Model loading, shared SQLite chunk loading, document extraction and indexing.

CHAPTER 6
RESULTS AND DISCUSSION

Current software evidence and outstanding empirical results

6.1 Implementation outcomes

Repository inspection confirms implementation of local file extraction, chunking, BM25 ranking, embedding and Chroma retrieval, paired comparison, search history, graded judgments, evaluation metrics, charts, and CSV export. The redesigned application was launched locally for interface review. The automated suite was run on 5 November 2026 with Python 3.10.0 and pytest 8.4.2; all 13 tests passed in 9.36 seconds. These are software verification results, not retrieval-quality results.

Table 6.1: Current implementation and evaluation status

Area	Status at report preparation	Evidence or next action
Application interface	Implemented and launched locally.	Streamlit app with guided left navigation and workspace pages.
Supported input formats	Implemented.	PDF, DOCX, TXT; scanned PDF OCR remains unavailable.
Paired retrieval workflow	Implemented.	Keyword and semantic methods share query, chunk collection, and K.
Evaluation workflow	Implemented; no completed study.	Create query set, judge passages, and run paired experiments.
Indexed corpus	Not present in current local database.	Choose and index a representative collection.
Relevance judgments	None recorded.	Grade every chunk for each study query and report coverage.
Quality and latency results	Not available.	Run experiments and export observed per-query measurements.
Automated test suite	13 passed; 0 failed.	Python 3.10.0; pytest 8.4.2; 9.36 seconds.

6.2 Functional test results

The executed pytest suite exercises core functions with controlled fixtures and an AppTest UI path. The passing result confirms that the covered code paths completed without assertion failures in this environment. The semantic retrieval test uses deterministic test vectors for the vector-store path; it does not load the production Sentence Transformer model or establish production relevance quality.

Table 6.2: Functional verification summary

Test area	Cases	Observed outcome
Application UI	1	Overview, upload/library flow, keyword search, and gated navigation to comparison rendered without exception.
Chunking	2	Chunk boundaries, overlap, metadata, and edge cases passed.
Persistence	1	Document/chunk records, search history, evaluation data, and deletion cascade passed.

Test area	Cases	Observed outcome
Document processing	4	TXT, DOCX/PDF extraction and expected empty, unsupported, corrupt/scanned input handling passed.
Evaluation metrics	2	Binary and graded metric calculations and empty-judgment behavior passed.
Keyword retrieval	2	Tokenization and BM25 ranking passed.
Vector retrieval	1	Deterministic-vector Chroma insert, query, and delete path passed.
Total	13	13 passed; 0 failed; 9.36 seconds.

6.3 Evaluation data status

The local SQLite database currently records no indexed documents, chunks, searches, test queries, relevance judgments, experiments, or evaluation results. As a result, there is no measured Precision@K, Recall@K, MRR, NDCG@K, or latency value to compare. No conclusion about retrieval quality or efficiency is supported at this stage.

6.4 Discussion

The software provides the controls needed to conduct a paired comparison, but implementation alone is not evidence that either method performs better. A meaningful result will depend on a representative collection, query categories that reflect the intended use, complete judgments, and consistent timing. The tool's distinction between retrieval-only and end-to-end query time should be preserved in any performance statement. In particular, a low Chroma query time does not include semantic query encoding, and neither timing scope includes initial model loading or document indexing.

The project is therefore at the application and protocol stage. Its strongest current result is the availability of a local, repeatable comparison workflow. The study's empirical claims remain open until the team supplies data and executes the evaluation protocol.

6.5 Current limitations

- No corpus or relevance judgments are present in the local data directory, so the comparison has no empirical results yet.
- The short starter corpus and query file are for demonstration and do not establish domain coverage or statistical representativeness.
- Incomplete labels can bias metrics because unjudged passages are treated as grade zero in the current metric calculation.
- Scanned PDFs require OCR, which the current document processor does not provide.
- BM25 rebuilds its in-memory index for a query; larger corpora may increase end-to-end keyword latency.
- SQLite and ChromaDB do not share an atomic transaction, so an interrupted indexing operation can require cleanup and re-indexing.
- The default model and tokenizer may not fit every domain or language; no model comparison has been performed. ChromaDB collection configuration should also be reviewed on dependency upgrades.

CHAPTER 7
CONCLUSION AND FUTURE ENHANCEMENT

What the current project establishes and what remains to be measured

7.1 Conclusion

Semantic Search Studio implements a local document-search application for comparing BM25 keyword ranking with embedding-based cosine retrieval. Its workflow covers upload and indexing, method-specific and paired search, query history, graded relevance judgments, ranking metrics, timing measurements, and experiment export. The code and documentation define a controlled protocol that can help the team compare lexical and semantic retrieval without assuming which approach will win.

The case study is not yet empirically complete. The current local database contains no corpus, judgments, or experiments, and the report therefore makes no claim about accuracy, speed, or superiority. The research questions can be answered only after the team selects a corpus, prepares categorized queries, judges relevant passages, and runs the paired protocol.

7.2 Future enhancement

1. Select a licensed, representative corpus and record its source, composition, and indexing configuration.
2. Prepare a balanced query set with paraphrase, exact-term, identifier, and domain-term categories.
3. Complete and review the 0-3 judgments for every corpus chunk; report coverage and assessor procedure.
4. Run paired experiments, export the query-level data, and replace this report's pending-results section with observed values and plots.
5. Record the test and runtime environment, then assess indexing and query performance at several corpus sizes.
6. Consider OCR for scanned PDFs, hybrid lexical-vector ranking, cached BM25 statistics, and a documented Chroma configuration migration.
7. Keep the RDF/OWL demonstration separate unless a future study explicitly defines a symbolic or ontology-aware retrieval method.

REFERENCES

[1] S. E. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009. doi: 10.1561/1500000019.

[2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of EMNLP-IJCNLP 2019, pp. 3982-3992. doi: 10.18653/v1/D19-1410.

[3] Sentence Transformers, “all-MiniLM-L6-v2 model card.” Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Accessed 4 November 2026.

[4] Chroma, “Collections - Chroma Cookbook.” Available: https://cookbook.chromadb.dev/core/collections/. Accessed 4 November 2026.

[5] Chroma, “Configuration - Chroma Cookbook.” Available: https://cookbook.chromadb.dev/core/configuration/. Accessed 4 November 2026.

[6] U.S. National Institute of Standards and Technology, “trec_eval: TREC evaluation software.” Available: https://github.com/usnistgov/trec_eval. Accessed 4 November 2026.

[7] Streamlit, “App model summary.” Available: https://docs.streamlit.io/get-started/fundamentals/summary. Accessed 4 November 2026.

[8] CzPhantom10, “semantic-search-case-study,” project source repository. Available: https://github.com/CzPhantom10/semantic-search-case-study. Accessed 5 November 2026.

APPENDIX A PROJECT SETUP AND CONFIGURATION

The project README targets 64-bit Python 3.10 or 3.11. The application is started from the repository root after creating a virtual environment and installing `requirements.txt`:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
streamlit run app.py
```

The first semantic request may download the configured model. The model can then run locally; CUDA is optional and CPU execution is supported. The runtime data directory defaults to `data/`, and the repository excludes runtime uploads and databases from version control.

Environment variables

Variable	Purpose
SEMSEARCH_DATA_DIR	Override the runtime data directory.
SEMSEARCH_MODEL	Choose the embedding model; re-index after changing it.
SEMSEARCH_CHUNK_SIZE	Set the word-count passage window.
SEMSEARCH_CHUNK_OVERLAP	Set the word-count overlap.
SEMSEARCH_TOP_K	Set the default result count.
SEMSEARCH_LATENCY_RUNS	Set the paired evaluation repeat count.
SEMSEARCH_MAX_UPLOAD_MB	Set the per-file upload limit.
SEMSEARCH_DEVICE	Choose CPU/CUDA or leave blank for automatic selection.

APPENDIX B STARTER QUERY EXAMPLES

The CSV below is included as a starting point for demonstrating the interface. These prompts do not constitute the final study query set and have not been judged against a corpus.

Table B.1: Starter query examples supplied with the project

Query	Category	Expected information
How can I protect my computer from hackers?	paraphrase	Network security techniques that prevent cyberattacks.
What security methods prevent cyberattacks?	lexical	Network security and protective techniques.
What does multi-factor authentication do?	specific concept	An authentication measure that helps protect accounts.

Before using these examples in a formal evaluation, review their categories, confirm that the intended answers are present in the selected corpus, and write additional queries that cover identifiers, exact terms, and paraphrases in a balanced way.

APPENDIX C SOURCE AND TEST INVENTORY

The suite was executed on 5 November 2026 with Python 3.10.0 and pytest 8.4.2. It completed with 13 passed and 0 failed in 9.36 seconds. Coverage is limited to the paths below; the suite does not measure retrieval effectiveness or production-scale performance.

Table C.1: Automated test modules in the repository

Module	Coverage indicated by the test file
test_app.py	Overview, upload/library display, keyword search, and comparison page rendering.
test_chunker.py	Chunk boundaries, overlap, metadata, and edge cases.
test_database.py	SQLite records, judgments, history, and evaluation persistence.
test_document_processor.py	TXT, DOCX, and PDF extraction, plus unsupported or empty inputs.
test_evaluation.py	Metric calculations for binary and graded relevance, including empty judgments.
test_keyword_search.py	BM25 tokenization and ranked keyword retrieval.
test_semantic_search.py	Vector-store interaction and semantic retrieval behavior.

The main entry point is `app.py`. Shared presentation elements are in `ui/`, core extraction and retrieval modules are in `core/`, orchestration is in `services/search_service.py`, and persistence is in `database/db_manager.py`. Project methodology, architecture, and evaluation instructions are in `docs/`.